

- LangChain is an **open-source framework** that helps developers build **LLM-powered applications** by connecting Large Language Models (LLMs) with external tools, data sources, memory, and workflows
- Instead of using an LLM alone, LangChain allows us to build real-world AI applications like chatbots, document Q&A systems, AI agents, and RAG applications.

🚩 Why do we use LangChain?

An LLM alone can only generate responses based on the data it was trained on.

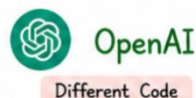
It **cannot**:

- Search the web
- Access private PDFs or documents
- Connect to databases
- Call APIs
- Use external tools
- Remember long conversations by itself

LangChain extends the capabilities of an LLM by allowing it to interact with these external resources.

Example: Suppose we want to use different LLM providers.

❌ Without LangChain



Each provider has its own SDK and syntax, so we have to write different code every time.



```
import openai
openai.api_key = "..."
response = openai.ChatCompletion.create(
    model="gpt-4o",
    messages=[{"role": "user",
                  "content": "Hello!"}]
)
print(response.choices[0].message.content)
```

```
import anthropic
client = anthropic.Anthropic(
    api_key="..."
)
message = client.messages.create(
    model="claude-3-opus",
    max_tokens=1000,
    messages=[{"role": "user",
                  "content": "Hello!"}]
)
print(message.content[0].text)
```

```
import mistralai
client = mistralai.Mistral(
    api_key="..."
)
chat_response = client.chat.complete(
    model="mistral-large-latest",
    messages=[{"role": "user",
                  "content": "Hello!"}]
)
print(chat_response.choices[0].message.content)
```

❌ Different libraries, different methods, different syntax.
More time writing code, less time building.

✅ With LangChain

We can use different LLMs with almost the same code.

Usually, we only need to change the model or provider.

✅ Same Code for All Providers

```
from langchain.chat_models import init_chat_model
# Just change the model name
model = init_chat_model("openai:gpt-4o") # or
# "anthropic:claude-3-opus"
# "mistral:mistral-large-latest"
response = model.invoke("Hello!")
print(response.content)
```



Benefits

- ✅ One unified interface
- ✅ Swap models easily
- ✅ Less code
- ✅ More time building real applications



LangChain acts as a bridge between your application and different LLM providers. You focus on **building**, not on handling different SDKs and syntaxes.

Components of LangChain

1 Models



The core intelligence behind any AI application.

Examples: OpenAI, Gemini, Anthropic, Mistral, etc.



2 Prompts



The instructions or input we give to the model. They tell the model what to do, how to respond, and in which format.



3 Chains

(LCEL in modern LangChain)



Chains combine multiple components (models, prompts, tools, etc.) in a sequence to solve a task. Modern LangChain uses LCEL (LangChain Expression Language) to build chains.



4 Memory



Gives the application the ability to remember information.

- Short-term memory (within session)
- Long-term memory (across sessions)

Helps in maintaining context in conversations.

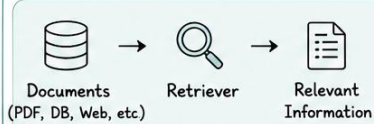


5 Retrieval

(Earlier called Indexes)



Helps the model find information from external data sources. Used in RAG (Retrieval Augmented Generation) to fetch relevant docs before generating the answer.

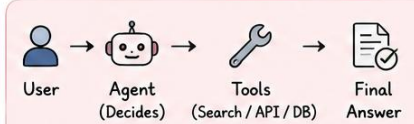


6 Agents



Agents can think, decide, and take actions using tools.

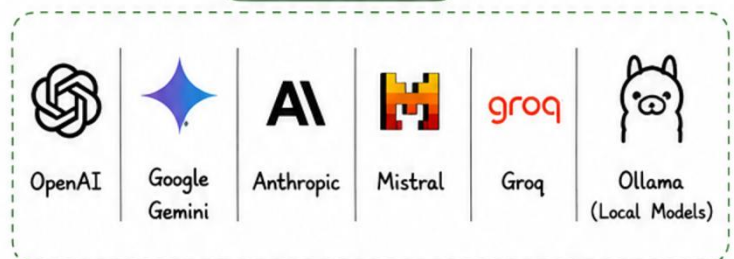
They can use external tools like search, APIs, calculators, databases, and more to get the job done.



1. Models :

- Models are the core intelligence of any AI application.
- They generate text, create embeddings, understand images, answer questions, and perform many other AI tasks.

Model Providers



Types of Models :

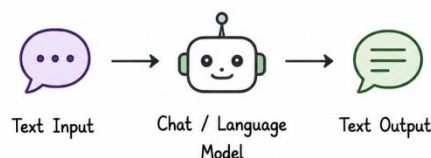
1. Chat / Language Models :

These models work with text.

They are mainly used to:

- Generate text
- Answer questions
- Write code
- Explain concepts
- Summarize content
- Translate languages

1. Chat / Language Models



- Generate text
- Explain concepts
- Answer questions
- Summarize content
- Write code
- Translate languages

2. Embedding Models :

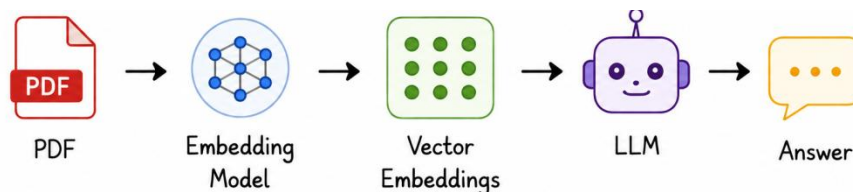
Embedding models convert text into numerical vectors. These vectors capture the meaning of the text rather than just the words.

Embedding models are mainly used for:

- Semantic Search
- Retrieval-Augmented Generation (RAG)
- Finding similar documents
- Recommendation systems

Example :

Suppose we have a PDF and we want an LLM to answer questions from it. The LLM cannot directly understand the PDF. So the process is:

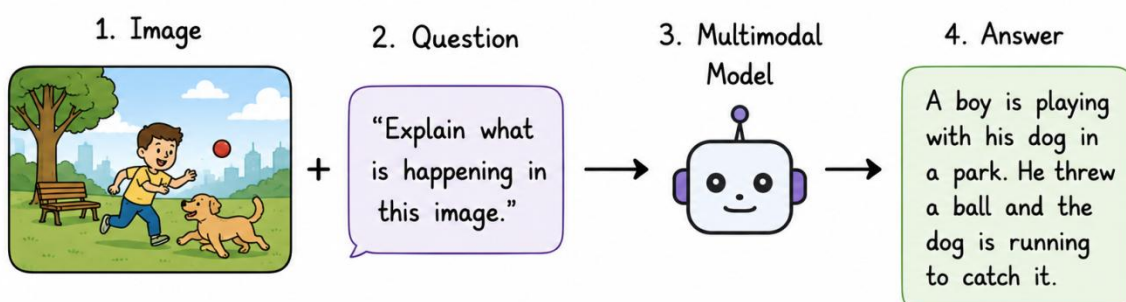


The embedding model converts the PDF into vectors first, then the LLM uses those vectors to retrieve relevant information before generating the answer.

3. Multimodal Models :

Multimodal models can understand more than just text. They can process:

- Images
- Audio
- Documents
- Videos (supported by some models)



2. Prompts :

- A prompt is the instruction or input we give to an AI model.

It tells the model:

- What to do
- How to respond
- Which format to use

Without a proper prompt, the model has no clear direction.

- Types of Prompts :

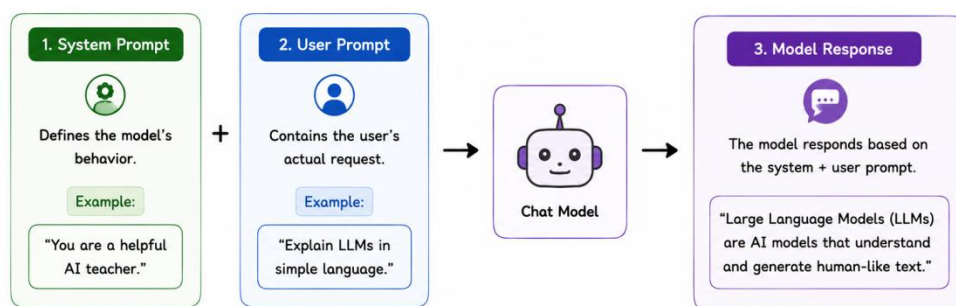
1. Simple Prompt :

A direct instruction given to the model.

Example: Explain what Machine Learning is.

2. System + User Prompt :

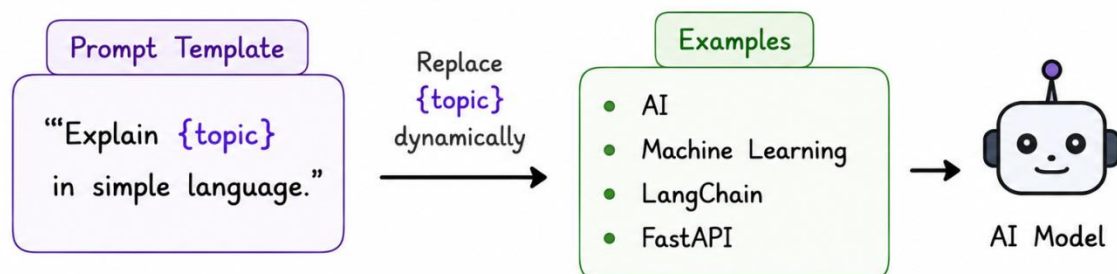
Modern chat models use different message roles.



The **system prompt** controls how the model responds, while the **user prompt** tells it what to answer.

3. Prompt Templates :

Instead of writing the same prompt repeatedly, we create reusable templates.



This makes prompts **reusable** and **easier to maintain**.

4. Structured Prompts :

Sometimes we want the output in a specific format.

Examples:

- JSON
- Bullet Points
- Tables
- Fixed Response Format

This helps when the output will be used by another application.

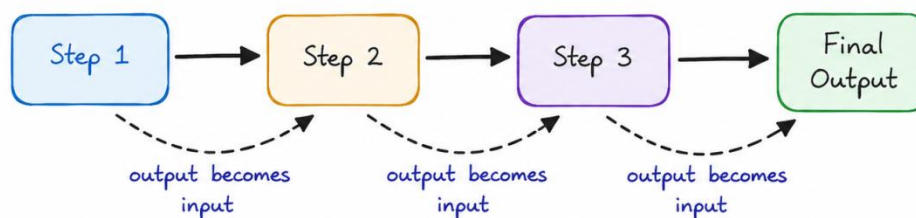
3. Chains :

A Chain is a sequence of steps connected together.

Instead of:



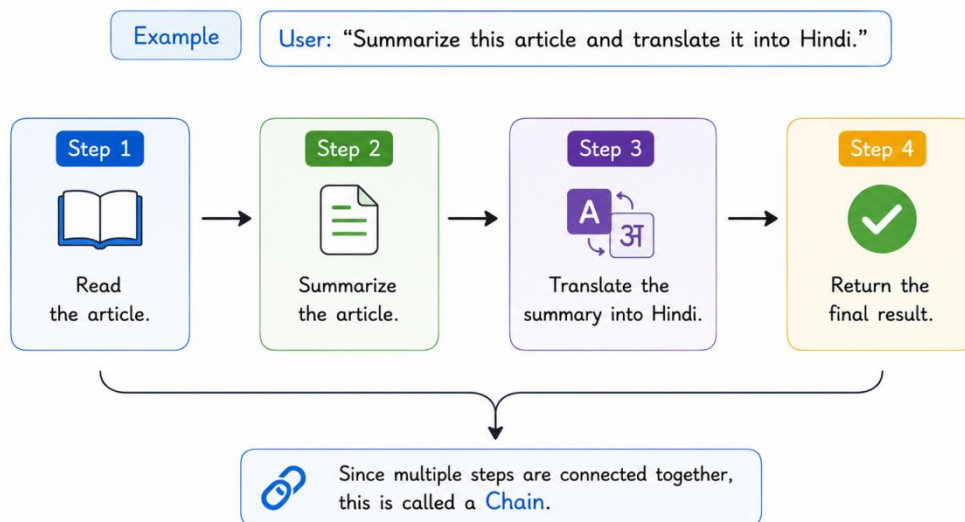
We create:



This complete workflow is called a **Chain**.

• Why do we use Chains?

- Real AI applications are usually not completed in a single step. Most applications need to perform multiple tasks before generating the final answer.
- Instead of writing separate code for every task, we connect all the steps together using a Chain.
- This makes the application easier to build, understand, and maintain.



- **Modern LangChain :**

Earlier versions of LangChain mainly used classes like:

- *LLMChain*
- *SequentialChain*

These two classes still exist in older projects, but new LangChain applications mainly use *LCEL*.

In modern LangChain, **LCEL (LangChain Expression Language)** is the recommended way to create chains.

Example:

```
chain = prompt | model
```

Here,

- **prompt** creates the input.
- **model** processes the prompt.
- The **response** is returned automatically.

The concept of a **Chain** is still the same—only the implementation has become simpler.

4. Memory :

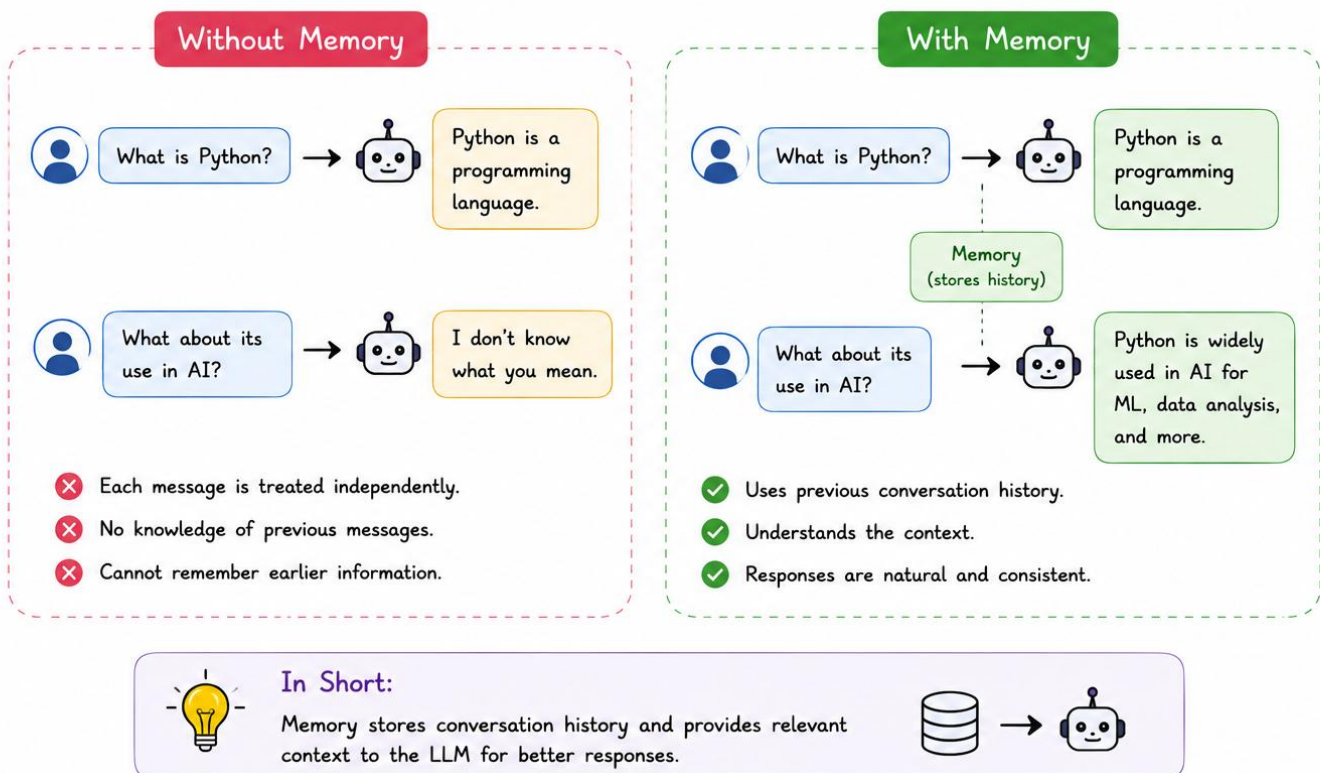
- Memory allows an AI application to remember previous conversations and use them to generate better responses.
- Without memory, every message is treated as a completely new request.
- With memory, the model can use previous conversation history to generate better and more natural responses.



Memory = Remember previous conversation



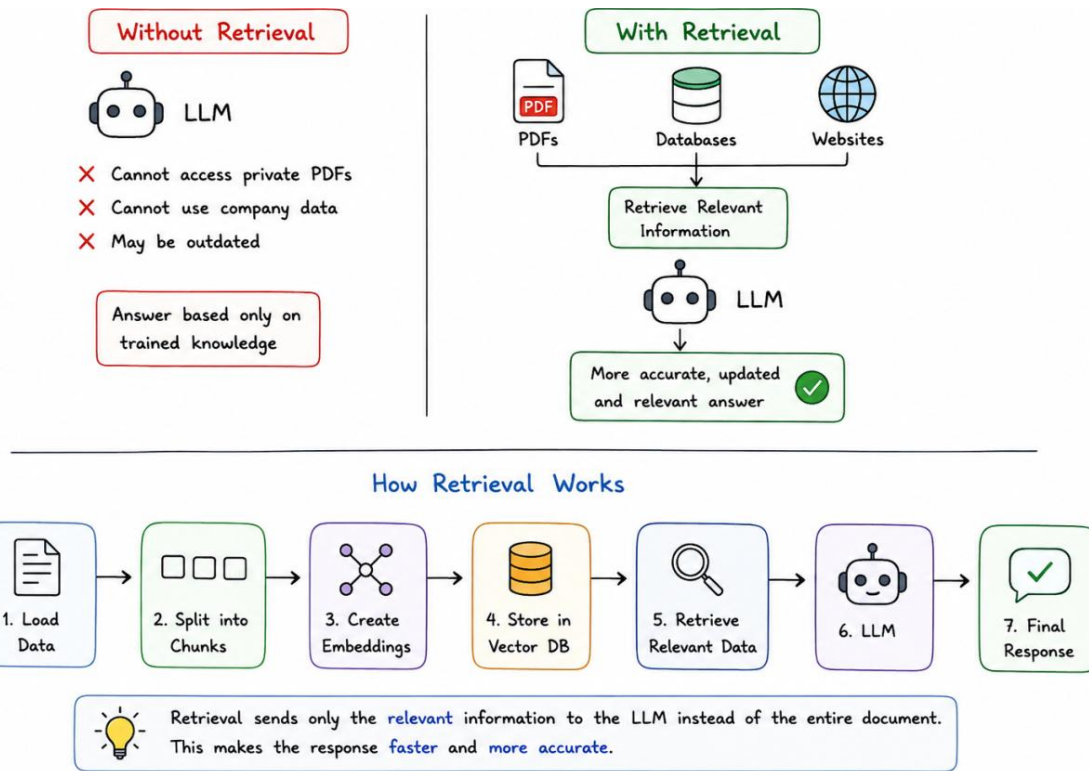
Better context → Better & natural responses



5. Indexes (Retrieval) :

- LLMs cannot directly access private files or real-time information.
- Indexes solve this by retrieving relevant external data and providing it to the LLM.

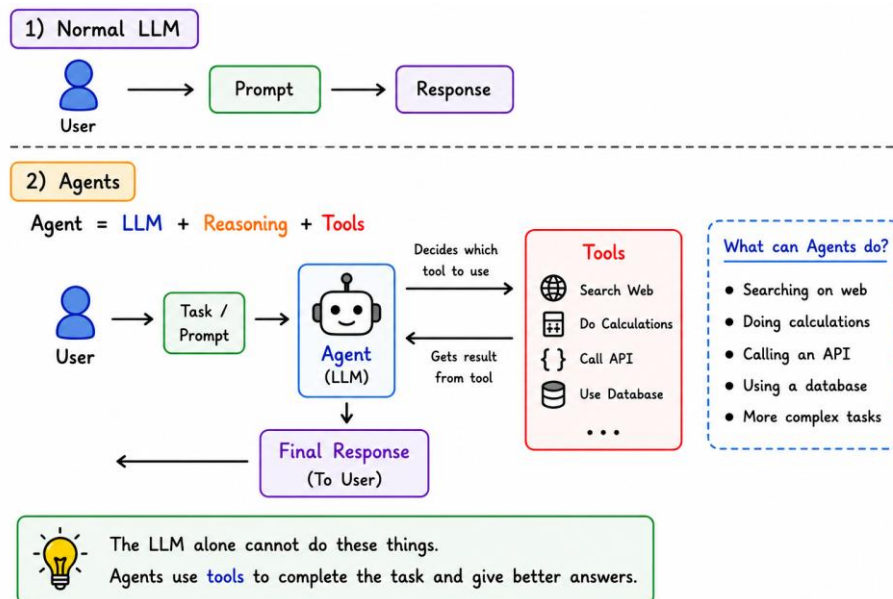
Note: In older versions of LangChain, this component was called Indexes. In modern LangChain, it is commonly referred to as Retrieval. Both represent the same concept.



6. Agents :

- An Agent is an AI system that can **reason**, **decide** what to do next, use tools if needed, and then give the final answer.
- Unlike a normal LLM, an Agent does not always follow fixed steps.
- It decides the next action based on the user's request.

Example :



• Tools :

A Tool is an external function that an Agent can use to complete a task. Examples: Search the Web, Calculator, Python, Weather API, Database, PDF Retriever. Tools help Agents perform actions beyond normal text generation.